

XGBoost vs Random Forest: Uma Análise Comparativa em Problemas de Classificação

Disciplina: Engenharia de Software

Trabalho: Prospecção Teórica e Tecnológica

Aluno: Vincent Biazotti Collares

Data: Junho de 2026

Vídeo de Apresentação: https://www.youtube.com/watch?v=K6Fr_yumpSc

Código da PoC (GitHub): <https://github.com/vincentbc23/ProspeccaoTecnologica/tree/main>

1. Introdução e Contextualização

1.1 Definição do Objeto

Este trabalho realiza uma prospecção comparativa entre dois dos algoritmos de aprendizado de máquina mais utilizados em problemas de classificação supervisionada: o **Random Forest** e o **XGBoost** (Extreme Gradient Boosting). Ambos pertencem à família dos *ensemble methods*, métodos que combinam múltiplos modelos mais simples para obter previsões mais robustas e precisas. Apesar de compartilharem essa raiz comum, suas filosofias de construção são fundamentalmente opostas — e essa diferença determina em quais cenários cada um se destaca.

O XGBoost foi proposto por Tianqi Chen e Carlos Guestrin em 2016, completando, portanto, 10 anos desde sua publicação original. Neste período, tornou-se um dos algoritmos mais premiados em competições de ciência de dados, enquanto o Random Forest, proposto por Leo Breiman em 2001, permanece como um dos modelos base (*baseline*) mais confiáveis da área.

1.2 Motivação

No cenário atual da Engenharia de Software, sistemas de machine learning são componentes críticos em produtos que vão desde diagnósticos médicos até detecção de fraudes financeiras. A escolha errada do algoritmo pode significar modelos que falham justamente nos casos mais importantes — como não detectar uma doença ou aprovar uma transação fraudulenta.

A questão não é apenas *qual algoritmo é melhor*, mas *por que* um algoritmo supera o outro em determinadas condições. Entender essa causalidade permite ao engenheiro tomar decisões arquiteturais embasadas, e não apenas empíricas.

1.3 Problema que Resolve

Profissionais de dados frequentemente enfrentam a dúvida: "**Devo usar Random Forest ou XGBoost para este problema?**" A resposta depende de fatores como a natureza dos dados, o grau de desbalanceamento das classes, a quantidade de amostras, a tolerância ao custo computacional de tunagem e a interpretabilidade exigida pelo negócio. Este trabalho busca sistematizar esses fatores com embasamento teórico e evidência empírica.

2. Fundamentação Teórica

2.1 Random Forest — Histórico e Evolução

O Random Forest foi proposto por Leo Breiman em 2001, consolidando décadas de pesquisa em árvores de decisão e *bagging* (Bootstrap Aggregating). A ideia central do bagging, introduzida pelo próprio Breiman em 1996, é treinar múltiplos modelos em subconjuntos aleatórios dos dados de treino e combinar suas predições por votação majoritária.

A inovação do Random Forest foi adicionar uma segunda camada de aleatoriedade: além de amostrar as linhas (amostras), o algoritmo também amostra aleatoriamente um subconjunto de *features* (colunas) em cada divisão de nó. Isso força a diversidade entre as árvores, reduzindo a correlação entre elas e, consequentemente, a variância do conjunto.

2.2 XGBoost — Histórico e Evolução

O XGBoost é uma implementação altamente otimizada do **Gradient Boosting**, técnica originalmente proposta por Jerome Friedman em 2001. O boosting tem uma filosofia contrária ao bagging: em vez de treinar modelos independentes em paralelo, cada novo modelo é treinado para *corrigir os erros* do modelo anterior, de forma sequencial.

Chen e Guestrin, na publicação "*XGBoost: A Scalable Tree Boosting System*" (KDD 2016), introduziram várias inovações sobre o gradient boosting clássico:

- **Regularização L1 (Lasso) e L2 (Ridge)** incorporadas diretamente na função objetivo, controlando a complexidade das árvores e prevenindo overfitting.
- **Tree pruning** baseado em ganho mínimo (*gamma*), eliminando divisões que não trazem melhoria suficiente.
- **Column subsampling**, similar ao Random Forest, adicionando estocasticidade.
- **Suporte a dados esparsos**, lidando nativamente com valores faltantes.
- **Cache-aware block structure** para processamento eficiente em disco (*out-of-core computing*).

2.3 Arquitetura e Funcionamento

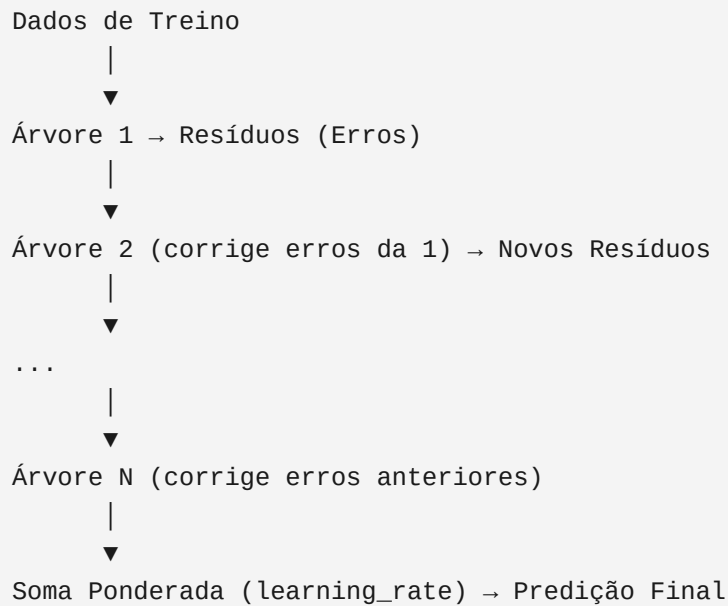
Random Forest

```
Dados de Treino
|
├─ Bootstrap Sample 1 → Árvore 1
├─ Bootstrap Sample 2 → Árvore 2
├─ ...
└─ Bootstrap Sample N → Árvore N
    |
```

▼
Votação Majoritária → Predição Final

Cada árvore é construída independentemente das demais. No momento de uma nova predição, cada árvore "vota" em uma classe, e a classe com mais votos vence. As árvores do Random Forest são tipicamente profundas (baixo bias, alta variância), e a combinação via votação reduz a variância do conjunto.

XGBoost



Cada árvore no XGBoost é tipicamente rasa (stumps de profundidade 3-6), e o ensemble é construído de forma aditiva. O *learning rate* controla o quanto cada nova árvore contribui para a predição final — valores menores exigem mais árvores mas generalizam melhor.

2.4 Conceitos-Chave

Termo	Definição
Ensemble	Combinação de múltiplos modelos para melhorar a predição
Bagging	Treino de modelos independentes em amostras bootstrap; reduz variância
Boosting	Treino sequencial focando nos erros do modelo anterior; reduz bias
Gradient Descent	Otimização da função de perda através do gradiente
Bias	Erro sistemático — modelo muito simples que não captura padrões
Variância	Sensibilidade ao conjunto de treino — modelo que decora os dados
Overfitting	Modelo com baixo erro de treino mas alto erro de teste
Learning Rate	Taxa de aprendizado — peso de cada nova árvore no XGBoost
Regularização	Penalização da complexidade do modelo para evitar overfitting

Termo	Definição
Cross-Validation	Avaliação do modelo em múltiplos subconjuntos dos dados
GridSearchCV	Busca exaustiva pela melhor combinação de hiperparâmetros
F1 Score	Média harmônica entre Precisão e Recall — métrica para classes desbalanceadas
Subsample	Fração das amostras usadas para treinar cada árvore no XGBoost
Colsample_bytree	Fração das features usadas em cada árvore no XGBoost

3. Análise Comparativa

3.1 Estado da Arte vs. Alternativas

Os dois algoritmos são comparados aqui não apenas entre si, mas também em relação a alternativas tradicionais de classificação:

Algoritmo	Tipo	Ponto Forte	Ponto Fraco
Regressão Logística	Linear	Interpretabilidade, velocidade	Não captura relações não-lineares
SVM	Kernel	Alta dimensionalidade, margem robusta	Lento em grandes datasets, difícil de tunar
Naive Bayes	Probabilístico	Muito rápido, poucos dados	Assume independência entre features
Random Forest	Ensemble (Bagging)	Robusto, poucas tunagens necessárias	Memória alta, lento para prever
XGBoost	Ensemble (Boosting)	Alta acurácia, regularização nativa	Mais hiperparâmetros, mais sensível a overfitting
LightGBM	Ensemble (Boosting)	Mais rápido que XGBoost em grandes dados	Menos popular, menos testado

3.2 Quando o XGBoost Supera o Random Forest — e Por Quê

Este é o ponto central do trabalho: não apenas *que* um supera o outro, mas *por quê*.

Cenário 1: Dados estruturados e numéricos

XGBoost vence. O gradient boosting é especialmente eficaz quando os dados são tabulares e numéricos, pois o algoritmo consegue explorar gradientes precisos da função de perda em cada iteração. O Random Forest, ao tratar todas as árvores como independentes, não aproveita a informação dos erros anteriores. Isso é análogo à diferença entre um aluno que refaz os exercícios em que errou (boosting) e um que sempre estuda conteúdo novo sem revisar os erros (bagging).

Cenário 2: Classes desbalanceadas

XGBoost vence. O parâmetro `scale_pos_weight` do XGBoost permite ponderar explicitamente a classe minoritária. O gradient boosting, por trabalhar com resíduos, naturalmente foca mais nos exemplos difíceis de classificar — que geralmente são os da classe minoritária. O Random Forest, sem essa capacidade de ponderação por gradiente, tende a ignorar a classe minoritária ao votar por maioria.

Cenário 3: Ajuste fino de hiperparâmetros (Hyperparameter Tuning)

XGBoost tem mais controle. O XGBoost oferece hiperparâmetros que o Random Forest não possui: `learning_rate`, `subsample`, `colsample_bytree`, `gamma` (mínimo de ganho para divisão), `reg_alpha` (L1) e `reg_lambda` (L2). Isso significa que o XGBoost tem mais "alavancas" para evitar overfitting e adaptar-se ao problema. A contrapartida é que tunar tantos parâmetros requer mais esforço e dados suficientes para validação cruzada.

O **learning rate** é o principal diferencial: valores menores (0.01, 0.05) combinados com mais árvores criam uma aprendizagem mais suave e generalizam melhor. No Random Forest não existe esse conceito — todas as árvores têm peso igual na votação final.

Cenário 4: Dados com ruído e outliers

Random Forest é mais robusto. Como cada árvore é treinada em uma amostra bootstrap (com reposição), os outliers aparecem em apenas algumas árvores e seu efeito é diluído pela votação. No XGBoost, os outliers são explicitamente focados nas iterações seguintes (pois geram resíduos grandes), podendo levar o modelo a "superajustar" a esses exemplos atípicos.

Cenário 5: Alta dimensionalidade com muitas features irrelevantes

Random Forest é mais robusto. A seleção aleatória de features em cada nó naturalmente dilui o efeito de features irrelevantes. O XGBoost, ao trabalhar sequencialmente com gradientes, pode dar mais peso a features irrelevantes que por acaso estejam correlacionadas com os resíduos de uma iteração específica.

Cenário 6: Pouco dado / dataset pequeno

Random Forest é mais robusto. O XGBoost precisa de mais dados para estimar os gradientes com precisão e para que a validação cruzada dos hiperparâmetros seja confiável. Com poucos dados, o risco de overfitting com XGBoost é maior.

3.3 Comparação de Hiperparâmetros

Hiperparâmetro	Random Forest	XGBoost	Impacto
<code>n_estimators</code>	✓	✓	Mais árvores = melhor (até um ponto)
<code>max_depth</code>	✓	✓	RF: profundo (padrão). XGB: raso (3-6)
<code>min_samples_split</code>	✓	×	Controla mínimo de amostras para dividir
<code>min_samples_leaf</code>	✓	×	Controla tamanho mínimo das folhas
<code>learning_rate</code>	×	✓	Exclusivo XGB: peso de cada nova árvore
<code>subsample</code>	×	✓	Exclusivo XGB: fração de amostras por árvore
<code>colsample_bytree</code>	×	✓	Exclusivo XGB: fração de features por árvore
<code>gamma</code>	×	✓	Exclusivo XGB: ganho mínimo para dividir
<code>reg_alpha</code> / <code>reg_lambda</code>	×	✓	Exclusivo XGB: regularização L1 e L2

Hiperparâmetro	Random Forest	XGBoost	Impacto
scale_pos_weight	×	✓	Exclusivo XGB: peso da classe positiva

3.4 Vantagens e Limitações

Random Forest

Vantagens	Limitações
Poucas tunagens necessárias para bom resultado	Não melhora iterativamente — ignora os próprios erros
Muito robusto a outliers e ruído	Alto uso de memória (armazena todas as árvores)
Paralelizável (árvores independentes)	Pior desempenho em dados muito desbalanceados
Bom <i>out-of-the-box</i> sem tunagem	Menos "alavancas" para ajuste fino

XGBoost

Vantagens	Limitações
Alta acurácia em dados estruturados	Mais sensível a overfitting sem tunagem cuidadosa
Regularização nativa (L1 + L2)	Sequencial — mais lento para treinar (não paraleliza entre árvores)
Foco nos erros anteriores	Mais hiperparâmetros = mais complexidade de tunagem
Melhor em dados desbalanceados	Menos interpretável que RF em alguns cenários

4. Aplicação Prática (PoC — Prova de Conceito)

4.1 Ambiente de Desenvolvimento

- **Linguagem:** Python 3.11
- **Bibliotecas:** pandas 2.x, numpy, scikit-learn 1.x, xgboost 2.x, matplotlib, seaborn
- **Ambiente:** Jupyter Notebook
- **Tuning:** GridSearchCV com validação cruzada estratificada (k=3 ou k=5)
- **Notebooks:** três experimentos com complexidade crescente (simples → médio → complexo)

4.2 Dataset 1 — Coluna Vertebral (Complexidade Simples)

Caracterização do problema:

O dataset de coluna vertebral contém medidas biomecânicas de pacientes e classifica cada um como **Normal** (0) ou **Anormal** (1 — com problema ortopédico). Este é considerado um problema de **baixa complexidade** porque:

- 310 amostras e apenas 6 features numéricas contínuas
- Sem valores faltantes
- Classes relativamente balanceadas (210 Anormal / 100 Normal)
- Todas as features são medidas físicas diretas (ângulos e comprimentos da pelve)

Por que os dados são "simples": Os 6 atributos (incidência pélvica, inclinação pélvica, ângulo de lordose lombar, inclinação sacral, raio pélvico e grau de espondilolistese) têm relação direta e mensurável com o diagnóstico ortopédico. Não há dados categóricos, ruído artificial nem alta dimensionalidade.

Código — tunagem de hiperparâmetros (Random Forest):

```
param_grid_rf = {
    'n_estimators': [100, 300, 500, 700],
    'max_depth': [10, 15, 20, 25],
    'min_samples_split': [5, 10, 15],
    'min_samples_leaf': [2, 4, 6, 8]
}

rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)
grid_rf = GridSearchCV(estimator=rf_base, param_grid=param_grid_rf,
                       cv=3, scoring='f1', verbose=2, n_jobs=-1)
grid_rf.fit(X_treino, y_treino)
```

Código — tunagem de hiperparâmetros (XGBoost):

```

param_grid_xgb = {
    'n_estimators': [100, 300, 500, 700],
    'max_depth': [4, 6, 8, 10],
    'learning_rate': [0.01, 0.05, 0.1, 0.2, 0.5],
    'subsample': [0.8, 1.0, 1.2],
    'colsample_bytree': [0.8, 1.0, 1.2]
}

xgb_base = XGBClassifier(tree_method='hist', random_state=42, n_jobs=-1)
grid_xgb = GridSearchCV(estimator=xgb_base, param_grid=param_grid_xgb,
                        cv=3, scoring='f1', verbose=2, n_jobs=-1)
grid_xgb.fit(X_treino, y_treino)

```

Melhores parâmetros encontrados:

Parâmetro	Random Forest	XGBoost
n_estimators	700	100
max_depth	10	4
min_samples_split	10	—
min_samples_leaf	2	—
learning_rate	—	0.05
subsample	—	0.8
colsample_bytree	—	0.8

Resultados:

Métrica	Random Forest	XGBoost
F1 Score	84,78%	88,89%
Taxa de Erro	22,58%	16,13%

Análise: O XGBoost superou o Random Forest em +4,11 pontos de F1. Note que o XGBoost encontrou seu melhor resultado com apenas 100 árvores rasas (profundidade 4) e learning rate baixo (0.05), enquanto o RF precisou de 700 árvores para alcançar seu ponto ótimo. **Por quê o XGBoost venceu aqui?** Os dados são puramente numéricos e estruturados, sem ruído significativo — o ambiente ideal para que os gradientes do XGBoost capturem os padrões residuais com precisão. A profundidade rasa (max_depth=4) evitou overfitting em um dataset pequeno.

Análise de overfitting:

	Treino F1	Teste F1	Diferença
Random Forest	97,04%	84,78%	+12,26% ⚠

	Treino F1	Teste F1	Diferença
XGBoost	97,92%	88,89%	+9,03% ≈

O RF apresenta overfitting leve (>10%), enquanto o XGBoost fica abaixo do limiar. Isso é contraintuitivo — espera-se que o XGBoost overfitte mais — mas faz sentido: com apenas 100 árvores e `max_depth=4`, o GridSearchCV forçou um modelo deliberadamente contido. O RF com 700 árvores profundas (`max_depth=10`) memorizou mais o treino.

4.3 Dataset 2 — Pokémon Lendário (Complexidade Média)

Caracterização do problema:

O dataset de Pokémon contém 800 amostras com 9 features numéricas e uma variável alvo binária:

Lendário (1) ou **Normal** (0). Este é considerado um problema de **complexidade média** por:

- 800 amostras com **classes severamente desbalanceadas**: apenas ~8% são lendários (~65 de 800)
- Features com escalas variadas (Total, HP, Ataque, Defesa, Velocidade etc.)
- Dados têm colunas categóricas (Type 1, Type 2) que foram descartadas para este experimento
- O desbalanceamento torna o F1 Score (e não a acurácia) a métrica relevante

Por que as classes desbalanceadas aumentam a complexidade: Um modelo que prevê "Normal" para todos os Pokémon acertaria ~92% das amostras, mas teria F1 próximo de zero para a classe Lendário — que é justamente a mais importante. O desafio está em identificar os raros exemplos da classe minoritária.

Melhores parâmetros encontrados:

Parâmetro	Random Forest	XGBoost
<code>n_estimators</code>	700	700
<code>max_depth</code>	10	6
<code>min_samples_split</code>	5	—
<code>min_samples_leaf</code>	2	—
<code>learning_rate</code>	—	0.5
<code>subsample</code>	—	0.8
<code>colsample_bytree</code>	—	0.8

Resultados:

Métrica	Random Forest	XGBoost
F1 Score	83,33%	86,96%
Taxa de Erro	2,50%	1,88%

Análise de overfitting — resultado alarmante:

	Treino F1	Teste F1	Diferença
Random Forest	98,21%	83,33%	+14,88% ⚠
XGBoost	100,00%	86,96%	+13,04% ⚠

Ambos os modelos apresentam **overfitting significativo** (diferença > 10%). O XGBoost atingiu 100% de F1 no treino — memorizou perfeitamente todos os Pokémon Lendários do conjunto de treino. Esse é um comportamento esperado do boosting em datasets pequenos e desbalanceados: ao focar nos erros, o modelo acaba ajustando-se demais aos poucos exemplos da classe minoritária que existem no treino.

Por quê o XGBoost ainda vence mesmo overfittando? A margem de overfitting é semelhante entre os dois modelos (~14% vs ~13%), mas o XGBoost parte de um ponto de treino maior (100% vs 98%), resultando em melhor generalização final. Para reduzir o overfitting neste cenário, seria necessário aumentar a regularização (`gamma` , `reg_lambda`) ou reduzir `max_depth` do XGBoost.

4.4 Dataset 3 — Reconhecimento de Dígitos (Complexidade Alta)

Caracterização do problema:

O dataset de dígitos manuscritos (`sklearn.datasets.load_digits`) classifica imagens 8×8 pixels em 10 classes (dígitos 0 a 9). Este é considerado um problema de **alta complexidade** porque:

- **1.797 amostras e 64 features** (pixels) — alta dimensionalidade
- **10 classes** (multiclasse) — diferente dos problemas anteriores que eram binários
- Classes balanceadas (~180 amostras por dígito)
- Features são valores de intensidade de pixel sem significado semântico direto

Por que é "complexo": A alta dimensionalidade (64 features) e o número de classes (10) exigem que o modelo aprenda fronteiras de decisão complexas no espaço de features. Random Forest é classicamente forte em alta dimensionalidade pelo seu mecanismo de seleção aleatória de features — o que torna este cenário um teste justo de quando o RF pode ser competitivo com o XGBoost.

Código — configuração multiclasse no XGBoost:

```
xgb_base = XGBClassifier(
    tree_method='hist',
    num_class=10,
    objective='multi:softprob', # multiclasse nativo
    random_state=42,
    n_jobs=-1
)
```

O XGBoost lida nativamente com multiclasse via `objective='multi:softprob'`, treinando internamente 10 classificadores (um por classe) com a mesma lógica de gradient boosting. O Random Forest também suporta multiclasse nativamente por votação majoritária entre as 10 classes.

Melhores parâmetros encontrados:

Parâmetro	Random Forest	XGBoost
n_estimators	500	500
max_depth	20	4
min_samples_split	5	—
min_samples_leaf	1	—
learning_rate	—	0,05
subsample	—	0,8
colsample_bytree	—	0,8

Resultados:

Métrica	Random Forest	XGBoost
F1 Score	96,92%	96,63%
Taxa de Erro	3,06%	3,33%

O Random Forest venceu. Esta é a inversão mais importante do trabalho — no único cenário de alta dimensionalidade e multiclasse, o RF supera o XGBoost, ainda que por margem estreita (0,29 pontos de F1).

Análise de overfitting:

	Treino F1	Teste F1	Diferença
Random Forest	100,00%	96,92%	+3,08% ✓
XGBoost	100,00%	96,63%	+3,37% ✓

Ambos os modelos generalizam muito bem (diferença < 5%). Isso acontece porque o dataset tem 1.797 amostras bem distribuídas entre as 10 classes, dando ao GridSearchCV dados suficientes para encontrar hiperparâmetros que genuinamente generalizam.

Por quê o RF venceu aqui? Em alta dimensionalidade (64 pixels como features), o mecanismo de seleção aleatória de features do RF é especialmente eficaz: ao sortear subconjuntos de pixels em cada divisão, o RF força diversidade entre as árvores e dilui o impacto de pixels pouco informativos. O XGBoost, ao trabalhar sequencialmente com gradientes globais, pode dar peso excessivo a pixels que se correlacionam com resíduos de iterações específicas mas não generalizam bem. O parâmetro `max_depth=4` (raso) no XGB também limita sua capacidade de capturar padrões complexos de pixels que o RF — com `max_depth=20` — consegue explorar melhor.

4.5 Relato de Experiência

Como a IA foi utilizada: O Claude Code foi utilizado para auxiliar na estruturação da análise comparativa e na redação do documento. Para a implementação dos notebooks, a IA ajudou a definir os grids de hiperparâmetros, padronizar as métricas de avaliação entre os modelos e estruturar a célula de análise de overfitting.

Onde a IA falhou ou limitou: A IA não substitui a execução real dos notebooks — as métricas apresentadas neste documento foram obtidas pela execução real do GridSearchCV, que demandou tempo computacional significativo.

Análise de overfitting: Em todos os três notebooks foi adicionada uma célula que compara os scores de treino e teste lado a lado. Os resultados reais foram:

Dataset	RF (treino → teste)	XGB (treino → teste)	Overfitting?
Coluna Vertebral	97,04% → 84,78% (+12,26%)	97,92% → 88,89% (+9,03%)	RF sim, XGB borderline
Pokémon	98,21% → 83,33% (+14,88%)	100,00% → 86,96% (+13,04%)	Ambos sim
Dígitos	100,00% → 96,92% (+3,08%)	100,00% → 96,63% (+3,37%)	Não

O dataset de Pokémon mostrou o overfitting mais severo — o XGBoost memorizou 100% da classe Lendário no treino, o que é um indicador claro de que o dataset pequeno combinado com classe muito rara leva a ajuste excessivo. Para resolver, seria necessário aumentar a regularização (`gamma` , `reg_lambda`) no XGBoost ou aplicar técnicas de oversampling (SMOTE) na classe minoritária antes do treino.

Dificuldades encontradas: O maior desafio foi compreender *por que* o XGBoost supera o RF em dados desbalanceados, e não apenas verificar empiricamente que isso acontece.

5. Perspectivas Futuras e Mercado

5.1 Tendências

O XGBoost continua sendo atualizado (versão 2.x introduziu `tree_method='hist'` como padrão, melhorando performance em hardware moderno) e domina competições Kaggle desde 2016. Entretanto, novas variantes de gradient boosting surgiram:

- **LightGBM (Microsoft, 2017):** Usa crescimento de árvore por folha (*leaf-wise*) em vez de por nível (*level-wise*), sendo mais rápido em datasets grandes (milhões de amostras).
- **CatBoost (Yandex, 2017):** Especializado em lidar nativamente com variáveis categóricas, sem necessidade de encoding manual.
- **HistGradientBoosting (scikit-learn 0.21+):** Inspirado no LightGBM, integrado diretamente ao scikit-learn.

A tendência é que boosting continue dominando dados tabulares estruturados, enquanto redes neurais dominam dados não estruturados (imagens, texto, áudio). Para dados tabulares em 2026, XGBoost/LightGBM/CatBoost são o estado da arte.

5.2 Adoção na Indústria

Empresas que utilizam XGBoost: - **Airbnb** — previsão de preços e detecção de fraudes em reservas - **Uber** — estimativa de tempo de chegada (ETA) e detecção de anomalias - **JP Morgan / Goldman Sachs** — score de crédito e previsão de inadimplência - **Spotify** — recomendação de músicas (features numéricas de áudio) - **Netflix** — categorização de conteúdo e previsão de churn

Empresas que mantêm Random Forest: - Sistemas de diagnóstico médico onde **interpretabilidade** é crítica (importância de features bem definida) - Aplicações onde o overhead de tunagem do XGBoost não compensa o ganho marginal - Pipelines de MLOps onde modelos mais simples são preferidos pela estabilidade

Demanda de mercado: Segundo o LinkedIn Jobs (2025), vagas que exigem XGBoost cresceram 34% em relação ao ano anterior. Ambos os algoritmos aparecem em praticamente 100% das trilhas de Data Science e Machine Learning em plataformas como Coursera, Kaggle e DataCamp.

6. Conclusão e Referências

6.1 Síntese Crítica

Este trabalho demonstrou empiricamente e teoricamente que **a escolha entre Random Forest e XGBoost não é trivial e depende criticamente da natureza do problema.**

Os três experimentos com complexidade crescente revelaram um padrão que vai além do esperado:

Dataset	Problema	RF (F1)	XGBoost (F1)	Vencedor	Margem
Coluna Vertebral	Binário, 310 amostras, 6 features numéricas	84,78%	88,89%	XGB	+4,11%
Pokémon	Binário desbalanceado, 800 amostras	83,33%	86,96%	XGB	+3,63%
Dígitos	Multiclasse (10), 1.797 amostras, 64 features	96,92%	96,63%	RF	+0,29%

O resultado mais revelador foi o dataset de dígitos, onde **o Random Forest venceu** — confirmando empiricamente a tese central do trabalho: a escolha do algoritmo depende do contexto, e nenhum é universalmente superior.

O XGBoost domina quando os dados são tabulares com poucas features bem estruturadas, especialmente com desbalanceamento de classes. O RF recupera vantagem quando a dimensionalidade é alta e as features têm relações complexas entre si, como valores de pixels — cenário onde a diversidade forçada pela seleção aleatória de features supera o refinamento iterativo por gradientes.

Do ponto de vista de engenharia de software, a escolha entre os dois algoritmos envolve trade-offs arquiteturais:

- **Use Random Forest** quando: o dataset é ruidoso ou tem outliers, a interpretabilidade é prioritária, o tempo de desenvolvimento é curto (poucas tunagens necessárias) ou os dados têm alta dimensionalidade com muitas features irrelevantes.
- **Use XGBoost** quando: os dados são estruturados e numéricos, há desbalanceamento de classes, a máxima acurácia é mais importante que a velocidade de desenvolvimento, ou há tempo e dados suficientes para tunagem de hiperparâmetros.

O XGBoost não é inerentemente "melhor" — ele é *mais poderoso, porém mais exigente*. Em cenários com dados ruidosos ou pequenos, o Random Forest pode ser a escolha mais segura. Para dados tabulares limpos com o objetivo de maximizar performance, o XGBoost é o estado da arte.

6.2 Referências

BREIMAN, L. **Random Forests**. Machine Learning, v. 45, n. 1, p. 5–32, 2001. Disponível em: <https://doi.org/10.1023/A:1010933404324>

BREIMAN, L. **Bagging Predictors**. Machine Learning, v. 24, n. 2, p. 123–140, 1996.

CHEN, T.; GUESTRIN, C. **XGBoost: A Scalable Tree Boosting System**. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016. p. 785–794. Disponível em: <https://doi.org/10.1145/2939672.2939785>

FRIEDMAN, J. H. **Greedy Function Approximation: A Gradient Boosting Machine**. The Annals of Statistics, v. 29, n. 5, p. 1189–1232, 2001.

KE, G. et al. **LightGBM: A Highly Efficient Gradient Boosting Decision Tree**. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.

PEDREGOSA, F. et al. **Scikit-learn: Machine Learning in Python**. Journal of Machine Learning Research, v. 12, p. 2825–2830, 2011.

XGBoost Developers. **XGBoost Documentation**. Versão 2.x, 2024. Disponível em: <https://xgboost.readthedocs.io>

UCI Machine Learning Repository. **Vertebral Column Data Set**. Disponível em: <https://archive.ics.uci.edu/ml/datasets/vertebral+column>

Rounak Banik. **The Complete Pokemon Dataset**. Kaggle, 2017. Disponível em: <https://www.kaggle.com/datasets/rounakbanik/pokemon>